

《网络安全技术与应用》

实验二：Socket 编程原理及实现

班级：计科 20215 班
完成地点：绿 2-102

姓名：卢瑶 学号：20211018341 时间：2024.3.28
指导教师：马洪亮

一、实验目的：掌握 Socket 编程原理，使用 TCP 和 UDP 协议的网络编程接口实现网络消息的发送和接收，并能进行一定的扩展。

二、实验环境：

1. 硬件环境：

- **计算机**：MacBook 搭载 Apple M1 芯片，具备高效的处理能力和优异的能效比，适合进行编程和网络分析。
- **网络连接**：确保 MacBook 连接到稳定的 Wi-Fi 网络，以便进行网络通信实验。

2. 软件环境：

- **操作系统**：macOS Monterey，优化了对 M1 芯片的支持，提供了更好的性能和安全性。
- **Python 环境**：Python 3.8

3. 开发环境：

- **PyCharm**：适用于 Python 的专业 IDE，具备代码完成、调试和项目管理等功能。确保下载适用于 M1 芯片的版本。

三、实验步骤：

1. 环境设置：

- 安装 Python 环境和 IDE。
- 确保计算机网络连接正常。
- （可选）安装 Wireshark 以便后续分析网络流量。

2. 理论学习：

- 复习 Socket 编程的基本概念，包括 TCP 和 UDP 的区别。
- 学习 Python 中 **socket** 模块的基本用法。

3. 编写 TCP 服务器和客户端：

- 开启 IDE，创建 Python 脚本用于 TCP 服务器。
- 编写代码以启动 TCP 服务器，监听特定端口。
- 创建另一个 Python 脚本作为 TCP 客户端，用于连接服务器并发送消息。
- 运行服务器和客户端，测试连通性和消息传递功能。
- 使用 **print** 语句在服务器和客户端显示发送和接收的消息。

4. 编写 UDP 服务器和客户端：

- 修改或创建新的脚本用于 UDP 通信。
- 设计 UDP 服务器以接收消息，并响应发送者。
- 创建 UDP 客户端，发送消息到服务器并接收回应。
- 运行并测试 UDP 通信效果。

5. 功能测试和错误处理：

- 对服务器和客户端进行各种情况下的测试，包括网络断开、消息丢

失等模拟环境。

- 编写错误处理代码，提高程序的健壮性。
6. **TCP 通信多线程扩展**：加入多线程模块，使得 TCP 通信支持多线程
 7. **结果记录和分析**：
 - 记录实验结果，包括成功和失败的测试。
 - 分析遇到的问题及解决方案。
 8. **撰写实验报告**：
 - 根据实验的结果和分析，撰写详细的实验报告。

实验 python 代码如下所示：

首先是 TCP_server 代码如下：📌

```
1.  import socket
2.
3.  def tcp_server():
4.      host = 'localhost'
5.      port = 12345
6.      server_socket = socket.socket(socket.AF_INET, socket.SOCK_STR
EAM)
7.      server_socket.bind((host, port))
8.      server_socket.listen(1)
9.      print('TCP Server is listening on port', port)
10.
11.     conn, addr = server_socket.accept()
12.     print('Connected by', addr)
13.
14.     try:
15.         while True:
16.             data = conn.recv(1024)
17.             if not data:
18.                 break
19.             print('Received:', data.decode())
20.             response = input("Enter your response: ") # 允许服务
器管理员输入响应
21.             conn.sendall(response.encode()) # 发送响应回客户端
22.         finally:
23.             conn.close()
24.
25. if __name__ == '__main__':
26.     tcp_server()
```

TCP_client 代码如下：📌

```

1.  import socket
2.
3.  def tcp_client():
4.      host = 'localhost' # 服务器地址
5.      port = 12345       # 服务器端口
6.
7.      # 创建 socket 对象
8.      client_socket = socket.socket(socket.AF_INET, socket.SOCK_STR
EAM)
9.      # 连接到服务器
10.     client_socket.connect((host, port))
11.
12.     message = 'Hello, TCP Server!'
13.     try:
14.         # 发送数据
15.         client_socket.sendall(message.encode())
16.         # 接收回响数据
17.         data = client_socket.recv(1024)
18.         print('Received:', data.decode())
19.     finally:
20.         # 关闭连接
21.         client_socket.close()
22.
23. if __name__ == '__main__':
24.     tcp_client()

```

UDP_server 代码如下：👉

```

1.  import socket
2.
3.  def udp_server():
4.      host = 'localhost'
5.      port = 12345
6.      server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGR
AM)
7.      server_socket.bind((host, port))
8.      print('UDP Server is listening on port', port)
9.
10.     try:
11.         while True:
12.             data, addr = server_socket.recvfrom(1024)
13.             print('Received from', addr, ':', data.decode())
14.             response = input("Enter your response: ")
15.             server_socket.sendto(response.encode(), addr)

```

```

16.         finally:
17.             server_socket.close()
18.
19.     if __name__ == '__main__':
20.         udp_server()

```

UDP_client 代码如下: 📌

```

1.     import socket
2.
3.     def udp_client():
4.         host = 'localhost'
5.         port = 12345
6.         client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
7.
8.         try:
9.             while True:
10.                 message = input("Enter your message: ")
11.                 if message.lower() == 'exit':
12.                     break
13.                 client_socket.sendto(message.encode(), (host, port))
14.                 data, server = client_socket.recvfrom(1024)
15.                 print('Received from server:', data.decode())
16.         finally:
17.             client_socket.close()
18.
19.     if __name__ == '__main__':
20.         udp_client()

```

多线程 TCP_server 代码如下: 📌

```

1.     import socket
2.     import threading
3.
4.     def handle_client(conn, addr):
5.         print(f"Connected by {addr}")
6.         try:
7.             while True:
8.                 data = conn.recv(1024)
9.                 if not data:
10.                     break
11.                 print(f"Received from {addr}: {data.decode()}")

```

```

12.         conn.sendall(data)
13.     finally:
14.         conn.close()
15.
16. def tcp_server():
17.     host = 'localhost'
18.     port = 12345
19.     server_socket = socket.socket(socket.AF_INET, socket.SOCK_STR
EAM)
20.     server_socket.bind((host, port))
21.     server_socket.listen()
22.     print('TCP Server is listening on port', port)
23.
24.     while True:
25.         conn, addr = server_socket.accept()
26.         thread = threading.Thread(target=handle_client, args=(con
n, addr))
27.         thread.start()
28.
29. if __name__ == '__main__':
30.     tcp_server()

```

四、实验结果：

4.1 TCP 协议通信结果

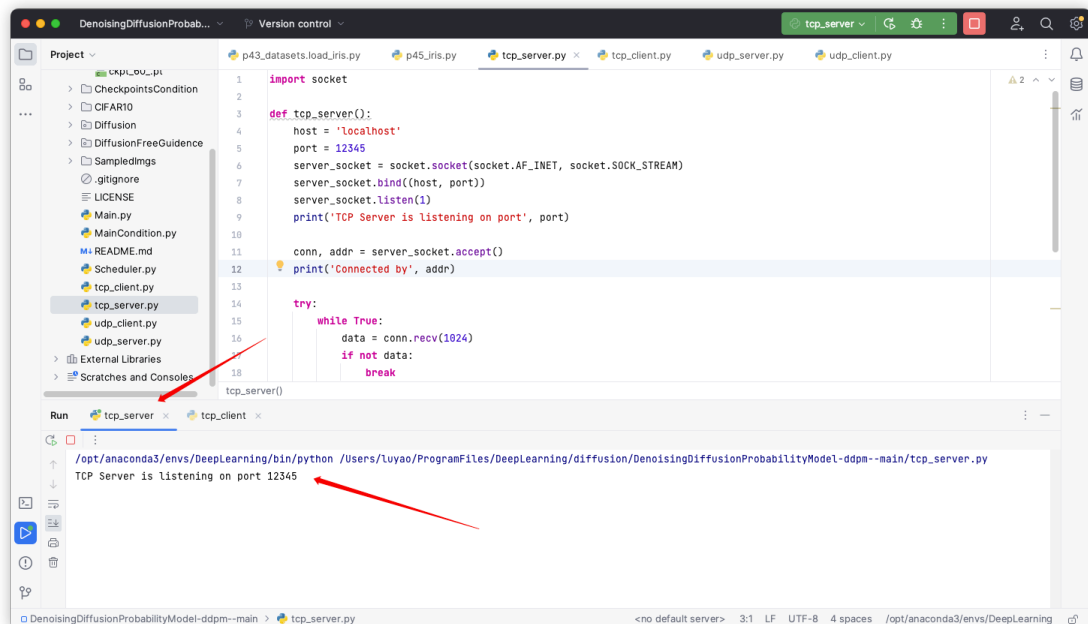


图 1 启动 TCP 服务器

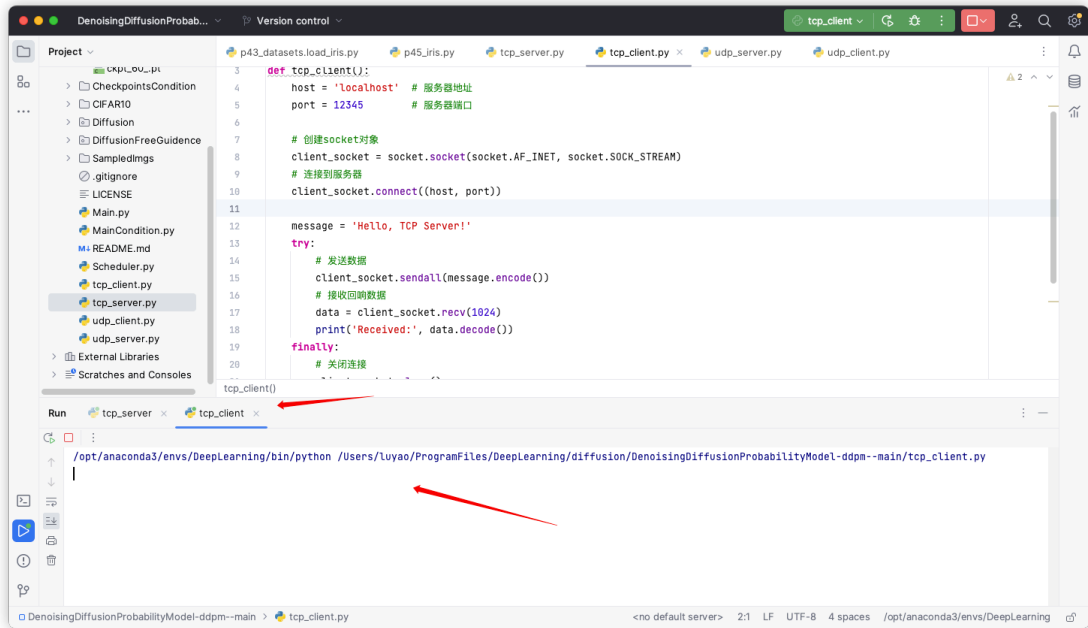


图 2 启动 TCP 客户端

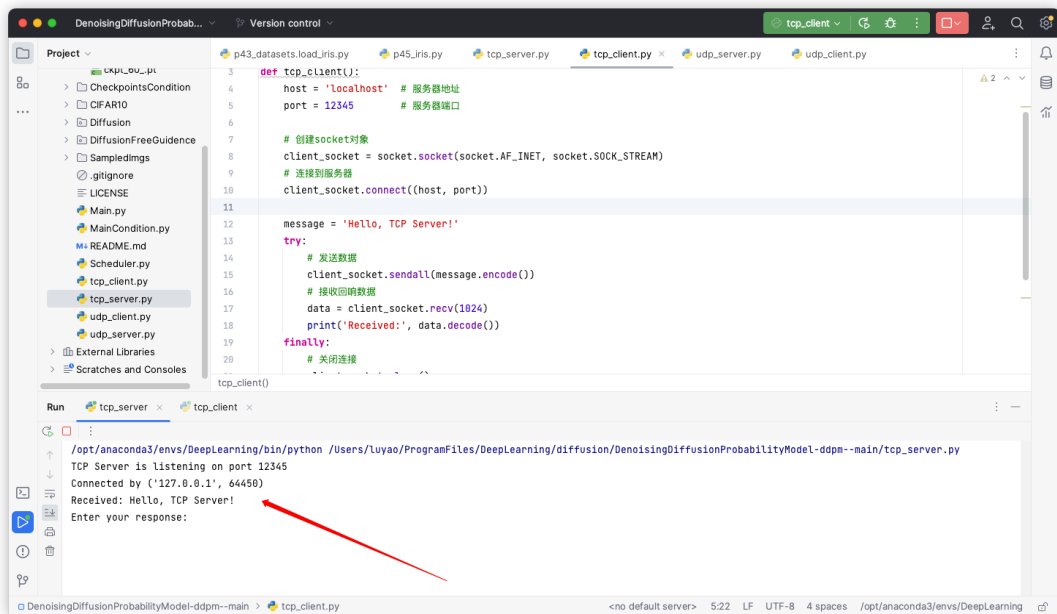


图 3 TCP 客户端发送信息

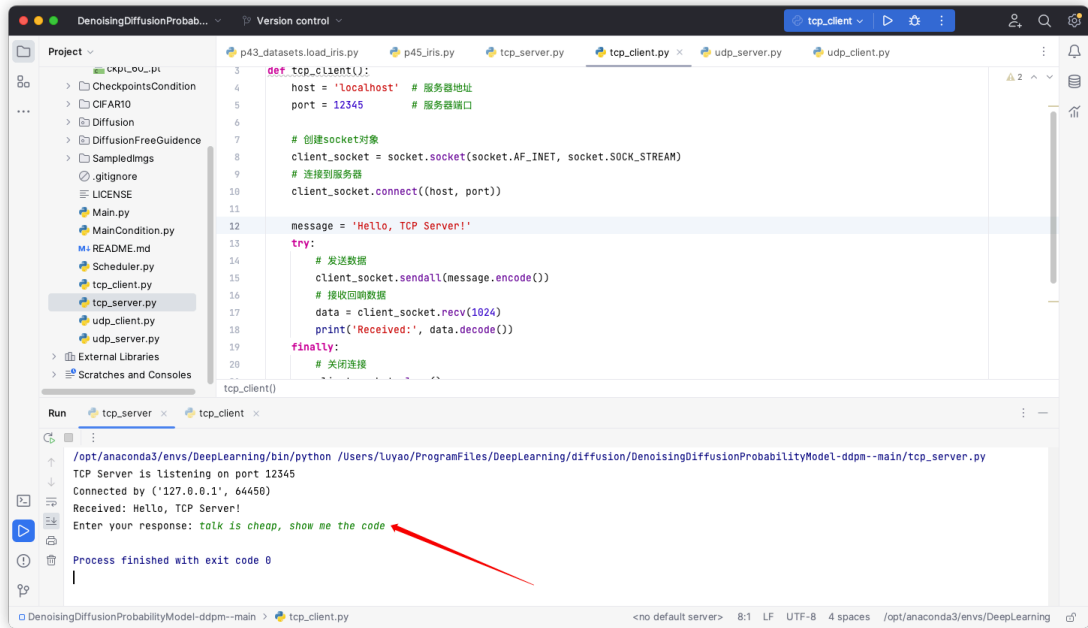


图 4 TCP 服务器成功接收信息

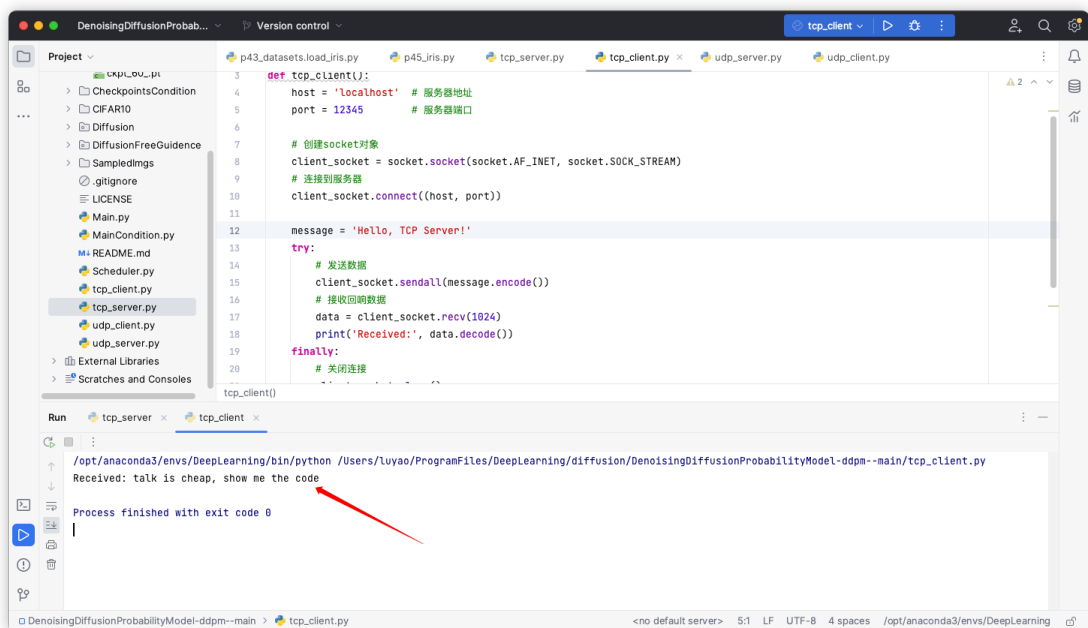


图 5 TCP 客户端接收成功

4.2 UDP 协议通信结果

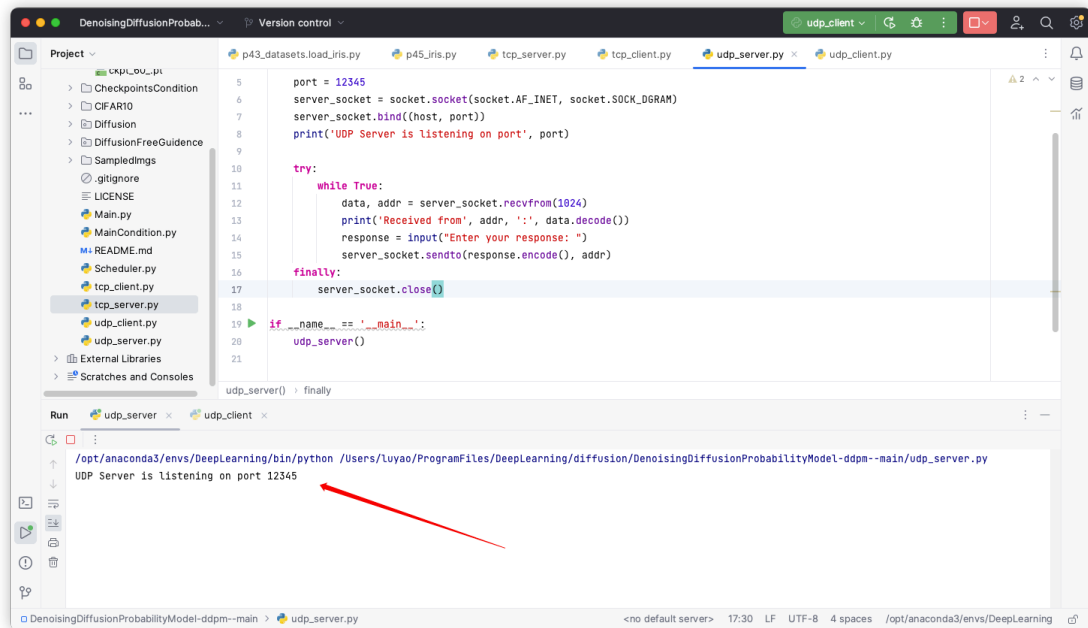


图 6 UDP 服务端启动

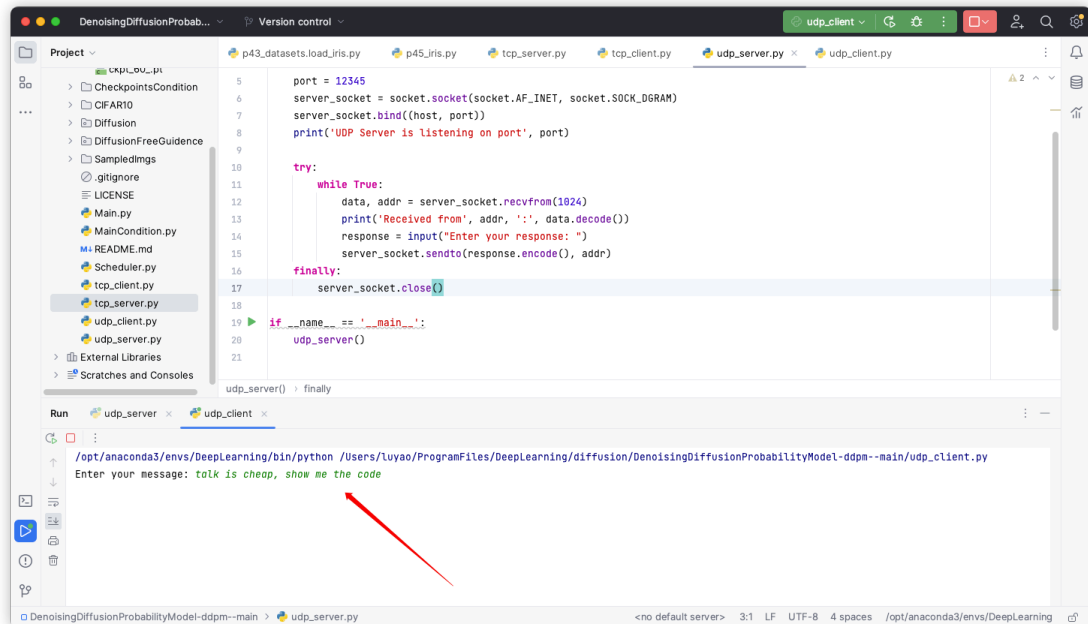


图 7 UDP 客户端发送消息

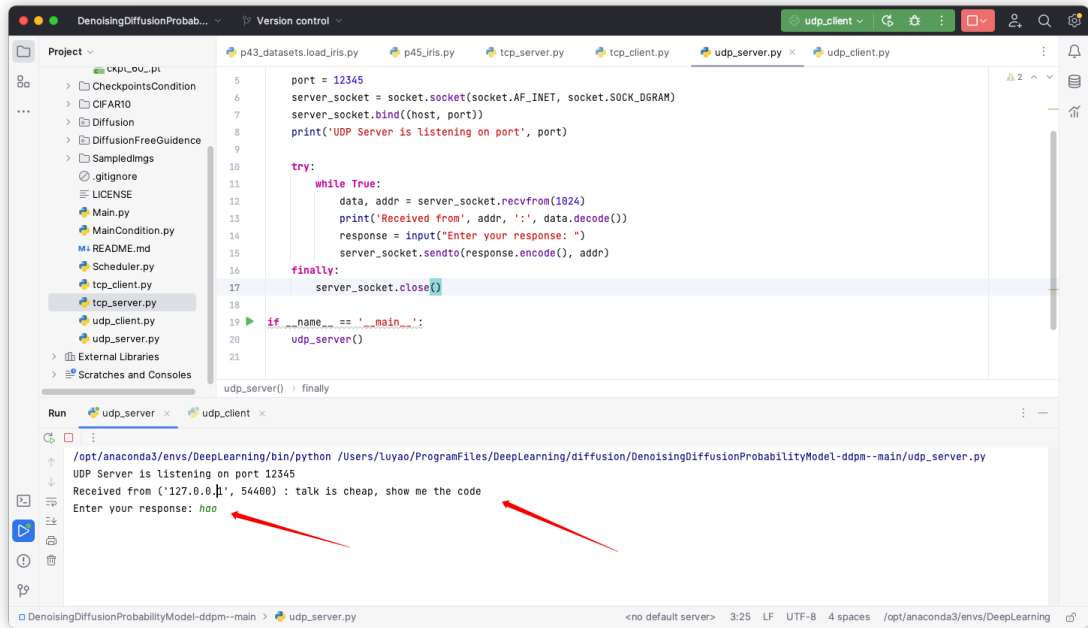


图 8 UDP 服务器接收消息并回应

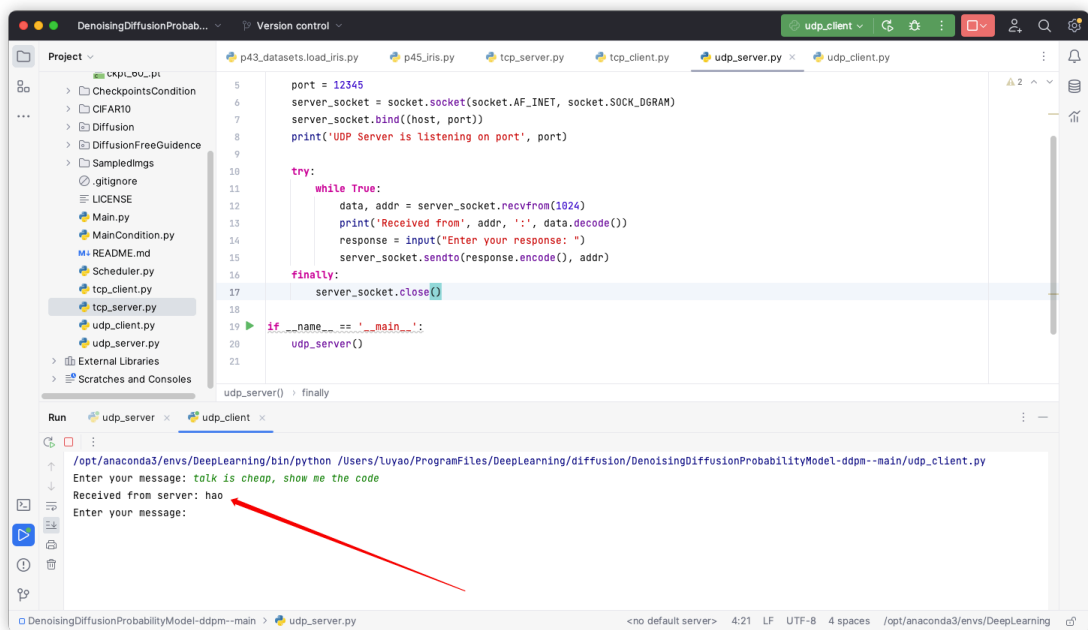
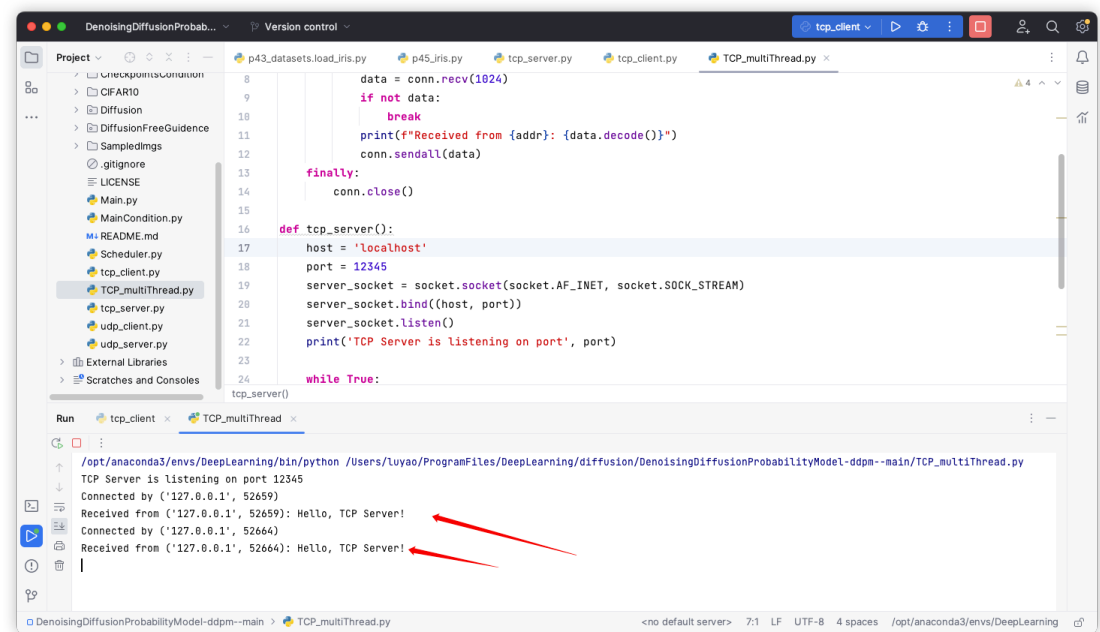


图 9 UDP 客户端成功接收消息

4.3 扩展：TCP 多线程通信



The screenshot shows an IDE with a project named 'DenoisingDiffusionProbab...'. The file explorer on the left lists several files, including 'TCP_multiThread.py'. The main editor displays the code for 'TCP_multiThread.py', which defines a 'tcp_server()' function. The function sets up a server on 'localhost' at port '12345', listens for connections, and enters a 'while True' loop to handle incoming data. The code includes comments and a 'finally' block to close the connection. The 'Run' tab at the bottom shows the execution output, which includes the message 'TCP Server is listening on port 12345' and two successful connections from '127.0.0.1' at ports 52659 and 52664, each receiving the message 'Hello, TCP Server!'. Red arrows point from the output messages to the corresponding lines in the code.

```
def tcp_server():
    host = 'localhost'
    port = 12345
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind((host, port))
    server_socket.listen()
    print('TCP Server is listening on port', port)
    while True:
        conn, addr = server_socket.accept()
        data = conn.recv(1024)
        if not data:
            break
        print(f'Received from {addr}: {data.decode()}')
        conn.sendall(data)
    finally:
        conn.close()

tcp_server()
```

Run: tcp_client x TCP_multiThread x

/opt/anaconda3/envs/DeepLearning/bin/python /Users/Luyao/ProgramFiles/DeepLearning/diffusion/DenoisingDiffusionProbabilityModel-ddpm--main/TCP_multiThread.py

TCP Server is listening on port 12345

Connected by ('127.0.0.1', 52659)

Received from ('127.0.0.1', 52659): Hello, TCP Server!

Connected by ('127.0.0.1', 52664)

Received from ('127.0.0.1', 52664): Hello, TCP Server!

图 10 TCP 多线程通信

五、实验结果分析:

通过这次实验，我发现 TCP 提供了一种**可靠的数据传输方式**，并成功基于多线程扩展了 TCP 通信过程，确保了数据正确性和顺序性，适合于对数据准确性要求高的应用。而 UDP 则提供了一种**简单的数据传输服务**，不保证数据的可靠性，但在某些需要高效传输的场合（如实时视频流）非常有用。在实际应用中，选择哪种协议取决于应用的具体需求。

六、实验心得:

通过这次 Socket 编程的实验，我不仅学会了如何使用 Python 进行网络编程，还深入理解了 TCP 和 UDP 这两种主要的网络协议的工作原理及其应用场景。实践中，我体会到了理论知识与实际操作之间的联系，加深了我的网络编程技能。此外，我也认识到了编程细节在网络通信中的重要性，比如在创建网络应用时如何选择合适的协议来满足需求。这次实验对于我的学习和未来的职业生涯都是一次宝贵的经验。